GET TECHY WITH LUCKY

TECH PULSE INSIDER

WEB DEVELOPMENT MASTERCLASS

COMPLETE LEARNING GUIDE

Git & GitHub

Version Control, Branching and the Professional Collaboration Workflow

WEEK 04 - JAVASCRIPT, GIT & GITHUB

The companion to the JavaScript Master Guide: how professional developers actually manage code.

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

Learn • Build • Connect • Grow

Course Information

Field	Detail
Program	Web Development Masterclass
Provider	Get Techy With Lucky / Tech Pulse Insider
Instructor	Lucky Nakola
Week	Week 04 of 08
Module	JavaScript + Git + GitHub
Estimated study time	8-10 hours
Builds on	Week 03 - Tailwind CSS and Modern Frontend Development
Format	Read -> Practise -> Quiz -> Build -> Submit
Assessment	Weekly quiz (auto-scored) and a practical assignment submitted via GitHub

Learning Objectives

By the end of this week you should be able to:

1. Explain what version control is and what problems it solves on real projects.
2. Use the everyday Git loop confidently: status, add, commit, push, pull.
3. Describe Git's three areas and explain why staging exists as a separate step.
4. Write commit messages a stranger could use to navigate your project's history.
5. Create, switch, merge and delete branches, and resolve merge conflicts calmly.
6. Collaborate on GitHub using remotes, pull requests and code review.
7. Keep secrets and dependencies out of a repository, and know what to do if one leaks.
8. Undo mistakes safely, and know which undo commands are dangerous on shared branches.

Prerequisites

Before starting this week, make sure you can already do the following:

- Build a working front-end project with HTML, CSS and JavaScript.
- Navigate folders and run commands in a terminal.
- Have Git installed and a GitHub account created.
- Understand what a text file is versus a binary file, and why that matters for diffs.

Table of Contents

This guide is written to be read in order. Each chapter builds on the one before it.

Chapter 1 - Introduction - The Problem Version Control Solves

Everyone invents version control before they learn it. The folder called project-final, then project-final-2, then project-final-REAL, then project-final-REAL-fixed. It works until it does not: you cannot remember which one had the working contact form, you cannot see what changed between two of them, and you certainly cannot let a second person work on the project at the same time.

Git solves all three. It records a full history of your project, lets you compare or return to any point in it, and lets several people work on the same codebase simultaneously without overwriting one another. It is the single most universal tool in professional software development. Every job you apply for will assume you know it.

WHY THIS MATTERS

Git is not backup, and it is not file sync. Its real value is that it makes changes reviewable and reversible. Once a change is a commit with a message, you can find when a bug appeared, understand why a decision was made, and undo exactly one thing without disturbing anything else.

Git and GitHub Are Not the Same Thing

	Git	GitHub
What it is	A program on your computer	A website that hosts Git repositories
Needs internet	No	Yes
Made by	Linus Torvalds, 2005	GitHub Inc, now Microsoft
Alternatives	Mercurial, SVN	GitLab, Bitbucket, Codeberg
Gives you	History, branches, merging	Hosting, pull requests, issues, CI, a portfolio

You can use Git with no internet connection and never touch GitHub. You cannot meaningfully use GitHub without Git. Learn Git first; GitHub is the collaboration layer on top.

How to Use This Guide

Read this guide with an editor open beside it. Type every example yourself rather than copying and pasting. Typing is slower, and that is the point: it is what turns a concept you have read into a skill you own. When something does not work, resist the urge to look at the answer straight away. Read the error, form a guess about the cause, then test that guess. Debugging is the skill that separates people who can build from people who can only follow.

Chapter 2 - How Git Actually Works: Snapshots, Not Differences

Most tools that track changes store a list of edits. Git does something different, and understanding this makes everything else easier. Each time you commit, Git records a complete snapshot of what every tracked file looked like at that moment. Files that did not change are not copied again - Git simply points at the previous version - so snapshots are cheap.

Each commit knows which commit came before it. That chain of snapshots is your history, and a branch is nothing more than a pointer to one commit in that chain. This is why branching in Git is instant: creating a branch writes a file containing a single commit hash.

```
A ---- B ---- C ---- D        <- main
                \
                 E ---- F      <- feature/contact-form

Each letter is a commit: a full snapshot plus a pointer to its parent.
A branch name is just a label pointing at one of them.
```

History is a graph of snapshots. Branches are labels on it.

First-Time Setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main

# Check what Git thinks:
git config --list
```

Do this once per machine. The name and email are stamped onto every commit you make.

NOTE

Use the same email address as your GitHub account, otherwise your commits will not be linked to your profile and your contribution graph will stay empty - which matters when a repository is doubling as your portfolio.

Chapter 3 - The Three Areas and the Everyday Loop

Area	What it holds	How things get there
Working directory	The files as they are on disk right now	You edit them
Staging area (index)	Changes selected for the next commit	git add
Repository (.git)	Permanent committed history	git commit

The staging area is the part beginners want to skip, and it is the part that makes Git good. It lets you fix three unrelated things in one session and still record them as three separate, meaningful commits, because you choose what goes into each one.

```
# What has changed?
git status

# Stage one specific file
git add styles.css

# Stage everything changed
git add .

# Record the staged changes
git commit -m "Add responsive card grid to services section"

# Send them to GitHub
git push origin main

# Bring down anyone else's work
git pull
```

The loop you will run hundreds of times. Learn these six commands properly and the rest can be looked up.

BEST PRACTICE

Run git status constantly - before adding, before committing, after merging, whenever you are unsure. It is free, it is safe, and it tells you exactly what Git is about to do. Most Git accidents happen because somebody ran a command without checking the state first.

Chapter 4 - Writing a Commit History Somebody Can Read

Your commit history is documentation. Six months from now, when a bug appears, somebody - probably you - will run git log looking for when a behaviour changed. A history of 'update', 'fix', 'changes' and 'asdf' is worthless for that.

Two Rules

- **One logical change per commit.** If your message needs the word 'and', it is probably two commits.
- **Write in the imperative.** 'Add contact form validation', not 'Added' or 'Adding'. It reads as an instruction the commit carries out.

Poor	Better	Why
update	Fix nav overlapping logo below 480px	Says what and where
fixed bug	Prevent double submit on contact form	Says which bug
stuff	Add dark mode toggle to site header	Findable in a search
final version	Replace placeholder copy with client text	Describes the change, not your mood

Poor	Better	Why
css	Extract card styles into reusable component	Explains the intent

Conventional Commits

Many teams prefix the subject with a type. It is not required, but it makes history scannable and it is what you will meet in industry.

```
feat:     a new feature            feat: add newsletter signup form
fix:      a bug fix                fix: correct total on cart page
docs:     documentation only       docs: explain env setup in README
style:    formatting, no logic     style: reformat with Prettier
refactor: restructure, no change   refactor: extract card component
test:     add or fix tests         test: cover email validation
chore:    tooling and maintenance  chore: bump tailwind to 3.4
```

The Conventional Commits convention. Pick a style with your team and stay consistent.

Chapter 5 - Branching and Merging

A branch lets you build something without touching the working version. The rule that makes this valuable is simple: main should always work. Anything experimental, half-finished or risky happens on a branch, and only joins main once it is ready.

```
# Create a branch and move onto it
git switch -c feature/contact-form

# ... work, add, commit as normal ...

# See all branches; the current one is marked
git branch

# Go back to main and bring the work in
git switch main
git merge feature/contact-form

# Tidy up once it is merged
git branch -d feature/contact-form
```

The full branch lifecycle. Older tutorials use 'git checkout -b'; 'switch' is the modern, clearer equivalent.

Naming Branches

- **feature/short-description** for new work
- **fix/short-description** for bug fixes
- **docs/short-description** for documentation
- **Keep them short, lowercase and hyphenated.** The branch name appears in every pull request and merge commit.

COMMON MISTAKE

Always create your branch from an up-to-date main. Run git switch main and git pull first. Branching from a stale main is the most reliable way to manufacture a painful conflict later.

Chapter 6 - Resolving Conflicts Without Panic

A conflict happens when two branches changed the same lines of the same file and Git refuses to guess which version is right. It is not an error and it is not your fault. It is Git correctly declining to make an editorial decision.

```
<<<<<<< HEAD
<h1>Welcome to Salon Bella</h1>
=======
<h1>Salon Bella - Nairobi</h1>
>>>>>>> feature/new-heading
```

Above the ======= is what your current branch says. Below it is what the incoming branch says.

9. Run git status to see exactly which files are conflicted.
10. Open each file and find the conflict markers.
11. Decide what the file should actually say. You may keep one side, the other, or write something new that combines both.
12. Delete all three marker lines. Leaving one in is the classic mistake and it will break your code.
13. Save, then git add the file to mark the conflict resolved.
14. Once every conflict is staged, run git commit to complete the merge.
15. Open the page in a browser and confirm it actually works before pushing.

BEST PRACTICE

If a merge goes badly wrong, git merge --abort returns you to exactly where you were before you started. Knowing this exists is what turns conflicts from frightening into routine.

Chapter 7 - GitHub: Remotes, Pull Requests and Collaboration

Connecting a Local Repository to GitHub

```
# Start tracking an existing project
git init
git add .
git commit -m "Initial commit"

# Point it at an empty GitHub repository
git remote add origin https://github.com/username/project.git
git branch -M main
git push -u origin main
```

```
# Or start from an existing remote
git clone https://github.com/username/project.git
```

The -u flag sets the upstream, so later you can just type git push.

The Pull Request Workflow

A pull request is a request to merge one branch into another, wrapped in a place to discuss it. It is where code review happens, and it is the standard way professional teams move work into main.

16. Update your local main: git switch main && git pull.
17. Create a feature branch from it.
18. Do the work in small, focused commits.
19. Push the branch: git push -u origin feature/whatever.
20. On GitHub, open a pull request. Describe what changed and why, not just what.
21. A teammate reviews it and leaves comments.
22. Respond with more commits on the same branch - the pull request updates automatically.
23. Once approved, merge it, then delete the branch.
24. Everyone else pulls the updated main.

WHY THIS MATTERS

The point of a pull request is not bureaucracy. It is that a second person catches things the author cannot see, that the discussion is recorded next to the code forever, and that main stays working because nothing reaches it unreviewed.

Writing a Pull Request Description

```
## What
Adds a dark mode toggle to the site header.

## Why
Requested by the client - most of their traffic is in the evening.

## How
Uses Tailwind's class-based dark mode. The choice is stored in
localStorage and falls back to the system preference.

## How to test
1. Load the page and click the moon icon in the header.
2. Reload - the choice should persist.
3. Clear localStorage and confirm it follows the OS setting.

Closes #14
```

A description that respects the reviewer's time. 'Closes #14' auto-closes that issue on merge.

Chapter 8 - The Files Every Repository Needs

.gitignore

```
# Dependencies - reinstallable, enormous, never commit
node_modules/
vendor/

# Secrets - never, under any circumstances
.env
.env.local
*.pem

# Build output - regenerated from source
dist/
build/

# Editor and OS noise
.vscode/
.idea/
.DS_Store
Thumbs.db

# Logs
*.log
```

A sensible starting .gitignore. Create it before your first commit, not after.

COMMON MISTAKE

Adding a file to .gitignore does not remove it from history if it was already committed. If you have committed a .env, the credentials inside it are compromised: rotate them immediately, then clean the history. Treat this as a security incident, not an untidy repository.

README.md

The README is the first and often only thing anybody reads. A useful one answers five questions.

- **What is this?** One or two sentences a stranger can understand.
- **What does it look like?** A screenshot or a live link.
- **How do I run it?** Exact commands, in order, that actually work on a clean machine.
- **How is it built?** The technologies used and roughly how the project is organised.
- **Who made it?** Your name, and a licence if the code is public.

Chapter 9 - Undoing Things Safely

Situation	Command	Safe on a shared branch?
Discard changes to one	git restore file.css	Yes

Situation	Command	Safe on a shared branch?
unstaged file		
Unstage a file, keep the edits	git restore --staged file.css	Yes
Fix the last commit message	git commit --amend	Only if not yet pushed
Undo a pushed commit	git revert <hash>	Yes - this is the right tool
Move the branch back, keep changes	git reset --soft <hash>	Only if not yet pushed
Move the branch back, discard changes	git reset --hard <hash>	No - destroys work
Shelve work temporarily	git stash / git stash pop	Yes
Abandon a merge in progress	git merge --abort	Yes

COMMON MISTAKE

git reset --hard permanently discards uncommitted work with no confirmation, and rewriting history that others have already pulled will break their clones. On anything shared, prefer git revert. It is the difference between correcting the record and forging it.

Chapter 10 - Important Terminology

Git's vocabulary is unusually precise, and being sloppy with it causes real confusion. 'Commit', 'push' and 'save' are three different things.

Term	Definition	In Plain Words	Example / Use Case
Version control	A system that records changes to files over time so any version can be recovered.	A time machine for your project.	Recovering the working version after a bad edit
Git	A distributed version control system that tracks project history locally.	The tool that records your snapshots.	git commit, git push
GitHub	A hosting service for Git repositories with collaboration features on top.	Where your repository lives online.	github.com/username/project
Repository	A project folder whose history Git is tracking.	Your project plus its whole history.	Created by git init
Local repository	The copy of the repository on your own machine.	Your copy.	The .git folder in your project
Remote	A named reference to a repository hosted elsewhere.	The online copy's address.	origin
origin	The conventional name for the remote you cloned from.	The default nickname for your GitHub copy.	git push origin main
Clone	Copying a remote repository, including its history, to your machine.	Download the project and its history.	git clone <url>
Working directory	The files as they currently exist on	What you are editing right	Your open editor tabs

Term	Definition	In Plain Words	Example / Use Case
	disk.	now.	
Staging area	The set of changes marked to go into the next commit.	The waiting room before a commit.	git add puts changes here
Commit	A permanent, named snapshot of the staged changes.	A save point with a message.	git commit -m "Add contact form"
Commit hash	The unique identifier Git gives each commit.	The commit's fingerprint.	a3f9c21
HEAD	A pointer to the commit you currently have checked out.	Where you are standing in history.	HEAD points at the tip of your branch
Branch	A movable pointer to a line of development.	A parallel version of the project.	main, feature/contact-form
main	The conventional name for the primary branch.	The trunk of the project.	Previously called master
Checkout / switch	Moving HEAD to a different branch or commit.	Jumping to another version.	git switch feature/nav
Merge	Combining the history of one branch into another.	Bringing work back together.	git merge feature/nav
Fast-forward merge	A merge where the target branch simply moves forward, no new commit needed.	No divergence, so just move the pointer.	Merging a branch when main has not moved
Merge commit	The commit created when two diverged branches are joined.	The knot tying two lines together.	Has two parent commits
Conflict	Git cannot decide automatically because both branches changed the same lines.	Two edits to the same line.	Git marks the file and asks you
Conflict markers	The <<<<<<<, ======= and >>>>>>> lines Git inserts into a conflicted file.	Git showing you both versions.	You edit them out by hand
Pull	Fetching remote changes and merging them into your branch.	Get everyone else's work.	git pull
Push	Sending your local commits to the remote.	Upload your work.	git push origin main
Fetch	Downloading remote changes without merging them.	Look before you leap.	git fetch origin
Fork	Your own copy of somebody else's repository on GitHub.	A personal copy of another project.	Used to contribute to open source
Pull request (PR)	A GitHub request to merge one branch into another, with review and discussion.	'Please review and merge my work.'	Opened from a feature branch
Code review	Another developer reading your changes before they are merged.	A second pair of eyes.	Comments on a pull request
Merge conflict resolution	Choosing what the combined file should say and committing that.	Deciding whose version wins, line by line.	Edit, stage, commit
.gitignore	A file listing paths Git should never track.	The 'do not record this' list.	node_modules/, .env
README.md	The Markdown file GitHub shows	The project's front door.	What it is, how to run it

Term	Definition	In Plain Words	Example / Use Case
	on a repository's front page.		
Markdown	A lightweight text format that renders as formatted documents.	Plain text that becomes headings and lists.	# Heading, **bold**
Untracked file	A file Git has never been told to record.	Git does not know about it yet.	Shown in red by git status
Staged change	A modification added to the staging area.	Ready to be committed.	Shown in green by git status
Diff	The line-by-line difference between two versions.	What exactly changed.	git diff
Log	The recorded history of commits.	The project's diary.	git log --oneline
Revert	Creating a new commit that undoes an earlier one.	Undo, but keep the record.	git revert a3f9c21
Reset	Moving the branch pointer, optionally discarding changes.	Rewind - dangerous if shared.	git reset --hard
Stash	Temporarily shelving uncommitted changes.	Put work aside for a moment.	git stash, git stash pop
Atomic commit	A commit containing one logical change and nothing else.	One idea per save point.	'Fix mobile nav overflow'
Feature branch	A short-lived branch for one piece of work.	A workspace for one task.	feature/dark-mode
Branch protection	GitHub rules preventing direct pushes to an important branch.	You must go through a pull request.	Protecting main
CI (continuous integration)	Automated checks that run on every push or pull request.	A robot that tests your work.	GitHub Actions running a build
Environment variable	A configuration value supplied outside the code.	A setting kept out of the source.	DATABASE_PASSWORD
Secret	A credential that must never be committed.	Passwords, API keys, tokens.	Kept in .env, listed in .gitignore
Commit message convention	An agreed format for writing commit subjects.	A house style for commit text.	feat:, fix:, docs:
Squash	Combining several commits into one when merging.	Tidying many small commits into one.	Squash and merge on GitHub
Open source	Software whose source is public and openly licensed.	Code anyone may read and use.	React, Linux, Git itself
License	The terms under which others may use your code.	The legal rules for your project.	MIT, Apache 2.0

Chapter 11 - Common Mistakes and How to Avoid Them

Mistake	What goes wrong	The fix
Committing .env or API keys	Credentials permanently exposed in history	Write .gitignore first; rotate any key that leaks
Committing node_modules	Enormous repository, endless	Ignore it - it is rebuilt by npm

Mistake	What goes wrong	The fix
	conflicts	install
One giant commit at the end	History has no useful detail	Commit each logical step as you finish it
Vague commit messages	Nobody can navigate the history later	Describe what changed and where
Working directly on main	No safe place to experiment; main breaks	Branch for every piece of work
Branching from a stale main	Avoidable conflicts at merge time	git switch main && git pull first
git pull with uncommitted work	Merge refuses or conflicts messily	Commit or stash before pulling
Leaving conflict markers in files	Syntax errors that reach production	Search for <<<<<<< before committing
git reset --hard to fix a pushed commit	Rewrites shared history and breaks clones	Use git revert
Never deleting merged branches	A branch list nobody can read	Delete after merge, locally and on GitHub

Chapter 12 - Security and Professional Considerations

Secrets Never Go in Git

This is the rule with the highest cost when broken. Bots continuously scan public GitHub for committed credentials, and a leaked cloud key can be found and used within minutes. Keep configuration in a .env file that is listed in .gitignore, and commit a .env.example with the variable names and no values so teammates know what to supply.

```
# .env  -- ignored, never committed
DB_PASSWORD=actual-secret-value
API_KEY=sk_live_realkey

# .env.example  -- committed, safe, documents what is needed
DB_PASSWORD=
API_KEY=
```

The pattern used by essentially every professional project.

Your Repositories Are Your Portfolio

For a developer with no formal work history, GitHub is the CV. An employer looking at your profile is reading your commit messages, your README quality, and whether your history looks like considered work or a single dump of files. Pin your best three or four repositories, give each a proper README with a screenshot and a live link, and keep the history clean.

- **Licence your public code.** Without a licence, others legally cannot use it. MIT is a simple, permissive default.
- **Never commit somebody else's credentials or client data.** Including screenshots containing real customer information.
- **Attribute borrowed code.** Using a snippet from Stack Overflow is fine; passing off a whole project as your own is not.
- **Enable two-factor authentication on GitHub.** Your account is the key to everything you have built.

Chapter 13 - Practical Exercise - Version Control Your Week 3 Project

25. Configure your name and email with git config if you have not already.
26. In your Week 3 Tailwind project, create a .gitignore covering node_modules, dist and .env.
27. Run git init, then git status, and read what it tells you.
28. Stage and commit everything as 'Initial commit'.
29. Create an empty repository on GitHub, add it as origin, and push.
30. Confirm on github.com that node_modules is not there. If it is, fix the .gitignore and remove it from tracking.
31. Create a branch feature/footer-links and add social links to the footer. Commit and push the branch.
32. Open a pull request for it, write a proper description, then merge it and delete the branch.
33. Locally: git switch main && git pull, and confirm the change arrived.
34. Manufacture a conflict deliberately: on main, change the hero heading and commit. Create a branch from the earlier commit, change the same line differently, then merge.
35. Resolve the conflict, remove every marker, and confirm the page still renders.
36. Run git log --oneline --graph --all and read your own history as a graph.
37. Write a real README with a description, screenshot, setup steps and your name.
38. Deliberately break something, commit it, then use git revert to undo it and observe that both commits remain in the log.

Chapter 14 - Knowledge Check - Weekly Quiz

Several of these have answers that feel wrong until you understand Git's model. If one surprises you, go back to Chapter 2 - almost every Git confusion traces to the snapshot idea.

Pass mark: 70% | *Answers are at the end of this chapter.*

Q1. What does git add actually do?

a) Saves your changes permanently
b) Moves changes into the staging area, ready for the next commit
c) Uploads your changes to GitHub
d) Creates a new branch

Q2. You are on main and run git switch -c feature/login. What happens?

a) A new branch is created and you move onto it
b) A new branch is created but you stay on main
c) main is renamed to feature/login
d) Your uncommitted changes are deleted

Q3. Which file stops node_modules and .env from ever being committed?

a) README.md
b) .gitignore
c) package.json
d) .gitkeep

Q4. What is the safest way to undo a commit that has already been pushed and that teammates have pulled?

a) git reset --hard and force push
b) git revert, which adds a new commit undoing the old one
c) Delete the repository and start again
d) Edit the commit message

Q5. A commit is a record of the differences since the last commit.

True / False

Q6. If you commit a password and then delete it in the next commit, the password is gone.

True / False

Q7. A merge conflict means you have done something wrong.

True / False

Q8. Explain the difference between the working directory, the staging area and the repository, using an everyday analogy.

Short answer - write two or three sentences.

Q9. Why is 'fix stuff' a poor commit message? Write a better version for a commit that corrects a navigation bar overlapping the logo on mobile.

Short answer - write two or three sentences.

Q10. Describe the pull request workflow in your own words, from starting a feature to it reaching main.

Short answer - write two or three sentences.

Answer Key

Mark your own work honestly - the goal is to find your gaps, not to score well.

#	Answer	Why
Q1	b - Moves changes into the staging area	add stages; commit saves; push uploads. Confusing these three is the single most common beginner error.
Q2	a - A new branch is	The -c flag creates the branch and checks it out in one step.

#	Answer	Why
	created and you move onto it	Uncommitted work comes with you.
Q3	b - .gitignore	Paths listed in .gitignore are never tracked. This is how you keep dependencies and secrets out of history.
Q4	b - git revert	revert adds history rather than rewriting it, so nobody else's clone breaks. Force-pushing a rewritten history over shared work is how teams lose commits.
Q5	False	Git stores full snapshots of the tree at each commit, reusing unchanged files by reference. Diffs are computed when you ask for them; they are not what is stored.
Q6	False	It remains in the history and can be read by anyone with the repository. The credential must be treated as compromised and rotated immediately.
Q7	False	A conflict simply means two branches changed the same lines and Git will not guess. It is a normal part of collaboration, and resolving one is a routine skill.
Q8	The working directory is your desk where you are actively working. The staging area is the envelope where you place the specific pages you want to file. The repository is the filing cabinet where sealed, dated envelopes are stored permanently.	The three-area model explains why add and commit are separate steps: staging lets you choose which of your current changes belong together in one commit.
Q9	It tells a future reader nothing about what changed or why, making history useless for finding when a bug appeared. Better: 'Fix mobile nav overlapping logo below 480px'.	Commit history is documentation. Its main value is being searchable six months later by someone who was not there.
Q10	Pull the latest main, create a feature branch, commit work in small logical steps, push the branch, open a pull request describing what and why, respond to review comments with further commits, then merge once approved and delete the branch.	This is the workflow essentially every professional team uses. Being able to describe it is the difference between using Git alone and working on a team.

Chapter 15 - Assignment - Collaborate on a Shared Repository

Objective

Work with a partner on a shared GitHub repository using branches and pull requests, so that you experience the full collaboration loop including at least one real merge conflict.

Scenario

You and one classmate are the two-person development team for a small community project - a local events noticeboard. You must both contribute features to the same codebase without overwriting each other's work, and every change must be reviewed before it reaches main.

Requirements

- One shared repository with both students added as collaborators.
- A README.md describing the project, both contributors, and how to run it.
- A .gitignore appropriate to the project.
- Branch protection on main so nothing can be pushed to it directly.
- Each student contributes at least three features, each on its own branch.
- Every branch reaches main through a pull request that the other student reviewed.
- At least one pull request contains a review comment that led to a follow-up commit.
- At least one genuine merge conflict, resolved and explained in the README.
- A commit history of small, atomic commits with descriptive messages.

Expected Output

A public GitHub repository whose Insights tab shows commits from both students, whose pull request list shows at least six merged pull requests with review activity, and whose main branch runs correctly when cloned fresh.

Technical Requirements

- Every feature developed on a branch named feature/<short-description>.
- No direct commits to main by anybody.
- Commit messages written in the imperative mood and under 72 characters in the subject line.
- No secrets, no node_modules and no build output committed at any point.
- Branches deleted after their pull request is merged.
- The README documents the conflict you hit, what caused it, and how you resolved it.

Submission Instructions

39. Ensure the repository is public and both students are listed as contributors.

40. Add a CONTRIBUTORS section to the README naming who built what.
41. Both students submit the same repository link on the Week 4 assignment page.
42. Each student additionally writes a short reflection in the submission notes: what went wrong, and what you would do differently.

Evaluation Criteria

Criterion	What Earns Full Marks	Weight
Branching discipline	All work on feature branches; main never pushed to directly	20%
Commit quality	Small, atomic commits with clear, searchable messages	20%
Pull request practice	Real reviews with comments that changed the code	20%
Conflict resolution	A genuine conflict resolved correctly and explained	15%
Repository hygiene	Good .gitignore, no secrets, clean structure, useful README	15%
Collaboration evidence	Both students contributed meaningfully and reviewed each other	10%

Bonus Challenges

- Add a GitHub Actions workflow that runs a linter on every pull request.
- Use issues to plan the work, and close them from commit messages with 'Closes #3'.
- Add a pull request template so every PR is described consistently.
- Protect main with a rule requiring at least one approving review before merge.

Chapter 16 - Summary

- Git records snapshots, not differences. A branch is a movable label pointing at one snapshot.
- Three areas: the working directory you edit, the staging area you curate, the repository that keeps history.
- Six commands cover almost everything: status, add, commit, push, pull, and switch.
- Commit small and often, and write messages a stranger could navigate by six months from now.
- main should always work. Everything else happens on a branch and arrives through a pull request.
- Conflicts are routine, not failures. Read both sides, decide, delete every marker, test, commit.
- revert adds history and is safe on shared branches; reset --hard destroys work and is not.
- Secrets never enter a repository. If one does, rotate it immediately - deleting it later is not enough.
- Your GitHub profile is read by employers. Treat READMEs and commit messages as part of the work.

You are not expected to remember everything in this guide. Professional developers look things up constantly. What you are expected to build is a mental model: knowing that a thing exists, roughly how it behaves, and where to look it up when you need the detail.

Further Reading and References

Documentation changes faster than any printed guide. Learning to read the official docs is part of the skill you are building.

- **Pro Git book - free and authoritative -** https://git-scm.com/book/en/v2
- **Git reference documentation -** https://git-scm.com/docs
- **GitHub Docs: getting started -** https://docs.github.com/en/get-started
- **GitHub Skills - interactive courses -** https://skills.github.com/
- **Conventional Commits specification -** https://www.conventionalcommits.org/
- **Oh Sh*t, Git!?! - recovering from common mistakes -** https://ohshitgit.com/
- **Learn Git Branching - visual practice tool -** https://learngitbranching.js.org/
- **Choose a License -** https://choosealicense.com/